



BRAINWARE UNIVERSITY

SCIENTIFIC CALCULATOR AND HOSPITAL APPOINTMENT SYSTEM

1. Basic Information

Programs Covered:	Program 1: Scientific Calculator Program 2: Hospital Appointment System
Name(s) of Student(s):	Soumyadwip Das (BWU/BTS/25/169) Ankita Paul (BWU/BTS/25/180) Sonia Naskar (BWU/BTS/25/195) Joyeta Mondal (BWU/BTS/25/214) Sayantan Bharati (BWU/BTS/25/503)
Programme & Semester:	B.Tech CSE & 3rd Semester
Course Name & Code:	Python Programming Lab (BES09013)
Name of the Guide/Supervisor:	MR. PRANAB GHARAI & MR. INDRANIL DAS
Project Date	11/09/2026





2. Introduction and Background

This project includes two modular Python applications: a Scientific Calculator for arithmetic and scientific functions, and a Hospital Appointment System for managing doctors, patients, and appointments. Both are menu-driven, use `main.py`, and store data in memory.

3. Objectives of the Project

Program 1 - Scientific Calculator

- Provide a menu of 13 operations: 7 two-number operations (add, subtract, multiply, divide, floor division, power, mod) and 6 one-number operations (sin, cos, tan, log, ln, sqrt)
- Perform trigonometric calculations in degrees and support logarithms with an optional base.
- Handle invalid inputs with clear error messages.
- Keep core math logic separate from input/output handling

Program 2 - Hospital Appointment System

- Register doctors (ID, name, specialty) and patients (ID, name, age, condition)
- View the lists of registered doctors, patients and appointments
- Book appointments only for existing patients and doctors, with unique auto-generated IDs.
- Separate models, in-memory storage and input/output into their own modules

4. Problem Statement / Driving Question

Program 1: How can a calculator separate reusable math logic from the menu while handling invalid inputs safely?

Program 2: How can a terminal system manage doctors, patients, and appointments while ensuring valid bookings?

5. Methodology

Program 1 - Scientific Calculator

- Split into 3 modules for math logic, input/output, and menu control.
- Created separate functions for each operation with safe error handling.
- Converted angles with `math.radians` so `sin`, `cos` and `tan` accept degrees
- Validated user input with `try/except`.

Program 2 - Hospital Appointment System

- Created Doctor, Patient, and Appointment classes with display methods.
- Centralised in-memory data storage with reusable helper functions.
- Added input validation and a shared Add/View/Back menu.
- Auto-generated IDs and verified Patient and Doctor IDs before booking.

6. Expected Outcomes / Deliverables

- Menu-driven Scientific Calculator with 13 operations and Exit option.
- Accurate results with clear error handling.
- Hospital Appointment System with Doctors, Patients, and Appointments menus.
- Valid appointments only for existing patients and doctors, using clean modular structures (3 and 6 modules).



7. Project File Structure

Program 1 - Scientific Calculator (3 modules)

```

.
├── main.py
├── calculator/
│   ├── __init__.py
│   ├── scientific_calculator.py
│   └── io.py

```

File	Responsibility
calculator/__init__.py	Package initialiser; re-exports everything from scientific_calculator and io
calculator/scientific_calculator.py	Core math logic: 7 two-number operations and 6 one-number operations
calculator/io.py	Input/output helpers: get2(), get1() and show()
main.py	Menu-driven entry point containing the MENU dictionary and main loop

Run with: `python main.py`

Program 2 - Hospital Appointment System (6 modules)

```

.
├── main.py
├── hospital/
│   ├── __init__.py
│   ├── doctor.py
│   ├── patient.py
│   ├── appointment.py
│   ├── data.py
│   └── io.py

```

File	Responsibility
hospital/__init__.py	Package initialiser; exports Doctor, Patient, Appointment and the data and io modules
hospital/doctor.py	Doctor model: id, name, specialty
hospital/patient.py	Patient model: id, name, age, condition
hospital/appointment.py	Appointment model: id, patient ID, doctor ID, date, time
hospital/data.py	In-memory lists (doctors, patients, appointments) with add, all_of and find helpers
hospital/io.py	Input/output helpers: inp, int_inp, show, show_list
main.py	Menu flow, ID counters and the add_doctor, add_patient, add_appt handlers

Run with: `python main.py`



8. Core Logic (Important Code)

Program 1 - Scientific Calculator

calculator/scientific_calculator.py – core math functions (excerpt)

```
import math

def add(a, b): return a + b
def division(a, b): return "Error: /0" if b == 0 else a / b
def mod(a, b): return "Error: %0" if b == 0 else a % b

def sin(a): return math.sin(math.radians(a))
def log(a, base=10):
    if a <= 0: return "Error: log(a<=0)"
    if base <= 0 or base == 1: return "Error: bad base"
    return math.log(a, base)
def ln(a): return "Error: ln(a<=0)" if a <= 0 else math.log(a)
def sqrt(a): return "Error: sqrt(-ve)" if a < 0 else math.sqrt(a)
```

calculator/io.py – input/output helpers

```
def get2():
    return float(input("First: ")), float(input("Second: "))

def get1():
    return float(input("Number: "))

def show(res, name):
    print(f"\n--- {name} = {res} ---")
```

main.py – menu table and dispatch (excerpt)

```
# (menu_no: label, func, needs_2_inputs)
MENU = {
    1: ("Add", c.add, 2), 2: ("Subtract", c.subtract, 2),
    # ... entries 3 to 10 ...
    11: ("Log", c.log, 1), 12: ("Ln", c.ln, 1), 13: ("Sqrt", c.sqrt, 1),
}

name, fn, n = MENU[ch]
try:
    if n == 2:
        a, b = get2()
        show(fn(a, b), name)
    elif ch == 11: # log with optional base
        a = get1()
        b = input("Base [10]: ") or 10
        show(fn(a, float(b)), name)
    else:
        show(fn(get1()), name)
except ValueError:
    print("Invalid number.")
```



Program 2 - Hospital Appointment System

hospital/doctor.py, patient.py, appointment.py – model classes (Doctor and Appointment shown)

```
class Doctor:
    def __init__(self, did, name, specialty):
        self.id, self.name, self.specialty = did, name, specialty
    def __str__(self):
        return f"[{self.id}] {self.name} ({self.specialty})"

class Appointment:
    def __init__(self, aid, pid, did, date, time):
        self.id, self.pid, self.did, self.date, self.time = aid, pid, did, date, time
    def __str__(self):
        return f"[{self.id}] P{self.pid} + D{self.did} @ {self.date} {self.time}"
```

hospital/data.py – in-memory storage and find helper

```
doctors, patients, appointments = [], [], []

def add(lst, item): lst.append(item)
def all_of(lst): return lst
def find(lst, _id): return next((x for x in lst if x.id == _id), None)
```

main.py – booking an appointment with ID validation

```
def add_appt():
    global aid
    p = data.find(data.patients, int_inp("Patient ID: "))
    if not p: return show("Patient not found.")
    d = data.find(data.doctors, int_inp("Doctor ID: "))
    if not d: return show("Doctor not found.")
    a = Appointment(aid, p.id, d.id, inp("Date: "), inp("Time: "))
    data.add(data.appointments, a); show(f"Booked ID={aid}"); aid += 1
```

main.py – reusable sub-menu for Doctors / Patients / Appointments

```
def manage(title, add_fn, lst):
    while True:
        print(f"\n--- {title} ---\n1. Add 2. View 9. Back")
        ch = int_inp("Choice: ")
        if ch == 1: add_fn()
        elif ch == 2:
            print("Empty." if not lst else "\n".join(str(x) for x in lst))
        elif ch == 9: break
        else: show("Invalid choice.")
```



9. Core Logic (Important Code)

Program 1 - Scientific Calculator	Program 2 - Hospital Appointment System
<pre>===== _ _ _ _ _ / \ / \ / \ / \ / \ --- --- --- --- --- ___ ___ ___ ___ ___ / \ / \ / \ / \ / \ _ _ _ _ _ ===== SCIENTIFIC CALCULATOR ===== --- CALCULATOR --- 1. Add 2. Subtract 3. Multiply 4. Divide 5. Floor Div 6. Power 7. Mod 8. Sin 9. Cos 10. Tan 11. Log 12. Ln 13. Sqrt 0. Exit Choice: 2 First: 56 Second: 32 --- Subtract = 24.0 --- --- CALCULATOR --- 1. Add 2. Subtract 3. Multiply 4. Divide 5. Floor Div 6. Power 7. Mod 8. Sin 9. Cos 10. Tan 11. Log 12. Ln 13. Sqrt 0. Exit Choice: </pre>	<pre>===== + + + + + --- --- --- --- --- --- --- --- + + + + + ===== HOSPITAL SYSTEM ===== === HOSPITAL === 1. Doctors 2. Patients 3. Appointments 0. Exit Choice: 1 --- Doctors --- 1. Add 2. View 9. Back Choice: 1 Name: sayantan Specialty: dentist -- Doctor added ID=1 -- --- Doctors --- 1. Add 2. View 9. Back Choice: 2 [1] sayantan (dentist) --- Doctors --- 1. Add 2. View 9. Back Choice: </pre>



10. Tools & Technologies Used

Tool / Technology	Purpose
Python 3.x	Core language for both console systems
math module	Trigonometric, logarithmic and square-root functions in the calculator
Modules and Packages	calculator and hospital packages split into focused modules via <code>__init__.py</code>
Classes (OOP)	Doctor, Patient and Appointment models with <code>__str__</code> display methods
Lists and Dictionaries	In-memory storage for hospital records; dictionary-driven calculator menu
Exception Handling	<code>try/except</code> <code>ValueError</code> and input re-prompting for invalid entries
String Formatting (f-strings)	Formatted menu, results and record display

11. References

- Python Official Documentation: <https://docs.python.org/3/>
- Python math Module Guide: <https://docs.python.org/3/library/math.html>
- Brainware University – Python Course Guidelines



12. Signatures

Signature of Student(s):

Date of Submission:

Signature of Guide/Supervisor:
